

CSCI 151 — Fall 2026

Programming for Scientist and Engineers

Introduction to the Course

Week 1 — Tuesday 18 August

Antonio Cerone

Talgat Manglayev

Adai Shomanov

Course Instructors

Lecture Instructors

- *Antonio Cerone* (Section 1)
- *Talgat Manglayev* (Sections 2 and 3)
- *Adai Shomanov* (Sections 4 and 5)

Lab Instructors

- *Iliyas Tursynbek* (Labs 1–5)
- *Olzhas Adiyatov* (Labs 6–10)

Teaching Assistants

- *TBA*

Course Overview

The course introduces students to

- structured programming
- the development of basic algorithms

using the C programming language

Course Overview

The course introduces students to

- structured programming
- the development of basic algorithms

using the C programming language

The course covers:

- language syntax

Course Overview

The course introduces students to

- structured programming
- the development of basic algorithms

using the C programming language

The course covers:

- language syntax and semantics
(in context: teaching by example)

Course Overview

The course introduces students to

- structured programming
- the development of basic algorithms

using the C programming language

The course covers:

- language syntax and semantics
(in context: teaching by example)
- control structures (e.g. loops & conditionals)

Course Overview

The course introduces students to

- structured programming
- the development of basic algorithms

using the C programming language

The course covers:

- language syntax and semantics
(in context: teaching by example)
- control structures (e.g. loops & conditionals)
- creation and use of basic data structures

Course Overview

The course introduces students to

- structured programming
- the development of basic algorithms

using the C programming language

The course covers:

- language syntax and semantics
(in context: teaching by example)
- control structures (e.g. loops & conditionals)
- creation and use of basic data structures
- sequential data types

Why the C Language?

- C allows you get a better understanding of **how computer memory works** in relation to programs (since requires that you manage memory yourself whereas newer languages do the memory management automatically)

Why the C Language?

- C allows you get a better understanding of **how computer memory works** in relation to programs (since requires that you manage memory yourself whereas newer languages do the memory management automatically)
- Hardware and device drivers are generally programmed using C (**robotics**) and libraries used by **mathematicians** and **physicists** are written in C and C++

Why the C Language?

- C allows you get a better understanding of **how computer memory works** in relation to programs (since requires that you manage memory yourself whereas newer languages do the memory management automatically)
- Hardware and device drivers are generally programmed using C (**robotics**) and libraries used by **mathematicians** and **physicists** are written in C and C++
- Newer languages like Java, C#, Python, etc. are actually **based on the C syntax**

The Flipped Classroom

Flipped classroom model

- most of the lesson materials will be provided via on-line readings and video tutorials

The Flipped Classroom

Flipped classroom model

- most of the lesson materials will be provided via on-line readings and video tutorials
- lecture time is meant for
 - Tuesday — in-class: recitation (on paper)
 - Thursday — online: revision and Q&A

The Flipped Classroom

Flipped classroom model

- most of the lesson materials will be provided via on-line readings and video tutorials
- lecture time is meant for
 - Tuesday — in-class: recitation (on paper)
 - Thursday — online: revision and Q&A
- lab time is for practice and is mandatory!

The Flipped Classroom

Flipped classroom model

- most of the lesson materials will be provided via on-line readings and video tutorials
- lecture time is meant for
 - Tuesday — in-class: recitation (on paper)
 - Thursday — online: revision and Q&A
- lab time is for practice and is mandatory!

It is critical that students work through the on-line materials regularly & keep up with the schedule.

The Flipped Classroom

Flipped classroom model

- most of the lesson materials will be provided via on-line readings and video tutorials
- lecture time is meant for
 - Tuesday — in-class: recitation (on paper)
 - Thursday — online: revision and Q&A
- lab time is for practice and is mandatory!

It is critical that students work through the on-line materials regularly & keep up with the schedule.

Office Hours: lectures and lab instructors will let you know how to registrar.

Course Website

<https://sst-csci.com/csci151/>

If you have not done it yet, **you must self register on the course website using your university email address**

Questions?

Communications

Office Hours

- for personal issues/questions and for discussing your code (**you cannot ask us to code for you, you can only discuss the code you have attempted to develop**)

Communications

Office Hours

- for personal issues/questions and for discussing your code (**you cannot ask us to code for you, you can only discuss the code you have attempted to develop**)

Discussion Forum on Piazza (<https://piazza.com/nu.edu.kz/fall2026/cs151>)

- **public posts** for issues/questions of general interest and for discussions among students (**code cannot be discussed or posted!**)

Communications

Office Hours

- for personal issues/questions and for discussing your code (**you cannot ask us to code for you, you can only discuss the code you have attempted to develop**)

Discussion Forum on Piazza <https://piazza.com/nu.edu.kz/fall2026/cs151>

- **public posts** for issues/questions of general interest and for discussions among students (**code cannot be discussed or posted!**)
- **private posts to 'All Instructors'** for urgent personal issues.

Communications

Office Hours

- for personal issues/questions and for discussing your code (**you cannot ask us to code for you, you can only discuss the code you have attempted to develop**)

Discussion Forum on Piazza (<https://piazza.com/nu.edu.kz/fall2026/cs151>)

- **public posts** for issues/questions of general interest and for discussions among students (**code cannot be discussed or posted!**)
- **private posts** to ‘All Instructors’ for urgent personal issues.

Do not use emails, use Piazza instead!

Graded Components

- 20% Lab Work and Programming Assignment
- 20% Midterm Exam 1 (Week 7)
Wed 30 Sep
(to be confirmed, time & room TBA)
- 20% Midterm Exam 2 (Week 11)
Fri 30 Oct
(to be confirmed, time & room TBA)
- 40% Final Exam (date, time, room TBA)

Unacceptable Communications

- Questions about grade curving
- Request to increase your grade (though you may inform the instructors and TAs if you believe that your grade is calculated wrongly)
- Complaints about the way the grade is calculated
- Requests to wave a penalty
- Requests to extend a deadline
- Requests for extra tasks

will NOT be answered!

Questions?

Live Grading

For any assessment you may be called for
live grading

- to explain your answers/code
- to rewrite part of your code
- to modify your code
- to write new code
- to answer questions on the material presented during lectures

Live Grading

For any assessment you may be called for
live grading

- to explain your answers/code
- to rewrite part of your code
- to modify your code
- to write new code
- to answer questions on the material presented during lectures

Your grade may be decreased after the live grading or even changed to zero

Code of Conduct

For any assessment:

- tasks **must be carried out individually**: do not make your code publicly available, do not show it to peers, do not post it anywhere (forums or repository such as GitHub, delete code from lab machines before leaving)

Code of Conduct

For any assessment:

- tasks **must be carried out individually**: do not make your code publicly available, do not show it to peers, do not post it anywhere (forums or repository such as GitHub, delete code from lab machines before leaving)
- **you are not allowed to submit LLM-generated code** (such as generated with **chatgpt**) for lab exercises and the programming assignment.

Code of Conduct

For any assessment:

- tasks **must be carried out individually**: do not make your code publicly available, do not show it to peers, do not post it anywhere (forums or repository such as GitHub, delete code from lab machines before leaving)
- **you are not allowed to submit LLM-generated code** (such as generated with **chatgpt**) for lab exercises and the programming assignment.
- the code you submit will be automatically checked for plagiarism (**do not copy, share, LLM-generate your code or buy solutions!**)

Academic Misconduct

Midterm Exams 1 and 2

- a 0 grade in the Midterm Exam

Academic Misconduct

Midterm Exams 1 and 2

- a 0 grade in the Midterm Exam

Lab Work and Programming Assignment

- a 0 grade in the Lab or programming Assignment

Academic Misconduct

Midterm Exams 1 and 2

- a 0 grade in the Midterm Exam

Lab Work and Programming Assignment

- a 0 grade in the Lab or programming Assignment

Final Exam: course failed!

Academic Misconduct

Midterm Exams 1 and 2

- a 0 grade in the Midterm Exam

Lab Work and Programming Assignment

- a 0 grade in the Lab or programming Assignment

Final Exam: course failed!

Moreover,

- all misconduct cases will be reported to the School
- repeated academic misconduct: course failed!

Questions?

How to Study and Practice

Videos

- Watch them before class

For today LESSONS 1–2 and for Thursday LESSON 3

- Do not watch in a passive way but stop the videos several times and use pen and paper

How to Study and Practice

Videos

- Watch them before class

For today LESSONS 1–2 and for Thursday LESSON 3

- Do not watch in a passive way but stop the videos several times and use pen and paper

Practice

- Do the **SELF QUIZ** and the **EXERCISES**

How to Study and Practice

Videos

- Watch them before class

For today LESSONS 1–2 and for Thursday LESSON 3

- Do not watch in a passive way but stop the videos several times and use pen and paper

Practice

- Do the **SELF QUIZ** and the **EXERCISES**
- Solve the problem **on paper before coding**

How to Study and Practice

Videos

- Watch them before class

For today LESSONS 1–2 and for Thursday LESSON 3

- Do not watch in a passive way but stop the videos several times and use pen and paper

Practice

- Do the SELF QUIZ and the EXERCISES
- Solve the problem on paper before coding
- Write also the code on paper

How to Study and Practice

Videos

- Watch them before class

For today LESSONS 1–2 and for Thursday LESSON 3

- Do not watch in a passive way but stop the videos several times and use pen and paper

Practice

- Do the SELF QUIZ and the EXERCISES
- Solve the problem on paper before coding
- Write also the code on paper
- Perform the code manually on paper

How to Study and Practice

Videos

- Watch them before class

For today LESSONS 1–2 and for Thursday LESSON 3

- Do not watch in a passive way but stop the videos several times and use pen and paper

Practice

- Do the **SELF QUIZ** and the **EXERCISES**
- Solve the problem **on paper before coding**
- Write **also** the code on paper
- Perform the code **manually on paper**
- Check the code on the computer

What NOT to do

Given how ‘forgiving’ compilers are (especially the online compilers), you may be tempted to do programming by **trial and error**, that is, by changing your code (more or less randomly) until you get the right result.

What NOT to do

Given how ‘forgiving’ compilers are (especially the online compilers), you may be tempted to do programming by **trial and error**, that is, by changing your code (more or less randomly) until you get the right result.

Warnings:

- programming by **trial and error** is likely to produce **wrong code**!

What NOT to do

Given how ‘forgiving’ compilers are (especially the online compilers), you may be tempted to do programming by **trial and error**, that is, by changing your code (more or less randomly) until you get the right result.

Warnings:

- programming by **trial and error** is likely to produce **wrong code**!
- if you get the right result but you do not know why, your code is likely to be **wrong**!

What NOT to do

Given how ‘forgiving’ compilers are (especially the online compilers), you may be tempted to do programming by **trial and error**, that is, by changing your code (more or less randomly) until you get the right result.

Warnings:

- programming by **trial and error** is likely to produce **wrong code**!
- if you get the right result but you do not know why, your code is likely to be **wrong**!

Advice: you must always **fully understand** what your program does!

Code Compilation

Some compilers forgive errors and allow code to compile also when it has critical errors.

Code Compilation

Some compilers forgive errors and allow code to compile also when it has critical errors.

Therefore:

- **you cannot claim** that your code compiles because it compiles on your machine

Code Compilation

Some compilers forgive errors and allow code to compile also when it has critical errors.

Therefore:

- **you cannot claim** that your code compiles because it compiles on your machine
- your programming assignments **must be compiled and tested on Moodle**

Code Compilation

Some compilers forgive errors and allow code to compile also when it has critical errors.

Therefore:

- **you cannot claim** that your code compiles because it compiles on your machine
- your programming assignments **must be compiled and tested on Moodle**

However, **you also cannot claim** that your code is correct if you get the expected results on the on-line compiler: some 'forgiving' compiler may even give the correct result in presence of errors!

Questions?

This Week's Lessons

Lesson 1: How do Programs Work? (no videos)

Lesson 2: Intro to Eclipse CDT

Lesson 3: Programs and Variables

Lesson 1

How do Programs Work? (no videos)

- Definitions
- Computer Hardware
- The CPU
- Low-level Languages
- High-level Languages
- The C Language

Questions?

Lesson 2

Intro to Eclipse CDT

- Developing Programs
- IDE Preliminaries
- Compilation and Build Settings
- Writing (Very Simple) Programs
- Comments

Questions?